

CPP NOTES – DAY 19

POINTERS

- Another powerful and important tool of C language. Reasons why pointers are used:
 1. enables us to access a variable that is defined outside the function.
 2. More efficient in handling data tables
 3. Reduce the length and complexity of program
 4. Increase the exec speed
 5. Use of pointer array to char strings results in saving a data storage space in memory.
- What is **Address**?: refers to location where data is staying.
- A pointer is a variable that stores the **memory address** of another variable. Rather than storing a direct value, it "points to" a location in memory where data is held.
- Declaration: `int *nameptr; , int *ptr = NULL (nullptr)`
- Assigning: `nameptr = &a` (address of a is assigned to nameptr)
- There are 4 types of pointers:
 1. Wild pointer: can access to anywhere (address).
`int *ptr; // wild pointer`
`*ptr = 10;`
 2. Generic pointer(Void pointer): can point to any data type. Often point to generic functions.
`void *ptr;`
`int x = 10;`
`ptr = &x;`
 3. Dangling pointer:
 4. Null pointer: Does not point to any memory location. Assigned to null at the time of declaration.
`int *ptr = NULL;`

DYNAMIC MEMORY ALLOCATION

In C and C++, dynamic memory allocation refers to **allocating memory at runtime** (during program execution) as opposed to **static memory allocation**, where memory is reserved at compile-time (like local variables or global variables). The need for dynamic memory allocation arises when we do not know the amount of memory required at compile-time or when the size of the data structures might change during execution.

The Functions/Operators in Dynamic Memory Allocation

1. malloc() (Memory Allocation)

- **Category:** C (standard library)
- **Purpose:** malloc is used to allocate a **block of memory** dynamically at runtime. It does **not initialize** the memory, meaning the content of the allocated memory is **garbage**.
- **Use Case:** When you need a specific amount of memory for variables or arrays, but the content doesn't need initialization.

2. calloc() (Contiguous Allocation)

- **Category:** C (standard library)
- **Purpose:** calloc allocates memory for an **array of elements** and initializes each element to **zero**. It is essentially a **combination of memory allocation and initialization**.
- **Use Case:** When you need both memory allocation **and initialization to zero** (common for arrays or structures).

3. realloc() (Reallocation)

- **Category:** C (standard library)
- **Purpose:** realloc is used to **resize a previously allocated block of memory**. This can be used to **expand** or **shrink** an already allocated block. It may also **move the memory** to a new location if there's not enough space in the current block.

- **Use Case:** When you need to change the size of a dynamic array or data structure while maintaining the existing content.

4. new (C++ Memory Allocation)

- **Category:** C++ (language operator)
- **Purpose:** new is a C++ operator that dynamically allocates memory for a single object or an array and **automatically calls the constructor** (if any) to initialize the object.
- **Use Case:** In C++, when you need to allocate memory for objects or arrays of objects, and you want the constructor to be called automatically.

FYI

- *Value of nullptr => (void *)0*
- *In the memory block of a variable, the content will be its value. But in the memory block of a pointer, the content will be the memory address of the variable it is pointing to.*
- *When I have ptr pointing to int , when tried to convert to char, it will cause error where you cannot change the datatype.*
- *Pointer cannot point to a value(cant hold value)*

- Flow of execution of arr pointers – $BA + (index\ value * sizeof\ data\ type)$
- `*ptr++` : print the value first then increment pointer(next location)
- `++*ptr`: value gets incremented
- `Malloc, calloc, realloc, new`
- `Malloc` retrurns the base address of the allocated memory if success.
- `Malloc, calloc, new` and `realloc` give unnamed addresses
- Better practice – first allocate then use it.